

# **Report 1: Inter-Process Communication Architecture**

uORB Messaging, XRCE-DDS Bridge, and AutosoarngControl Message  
Design

PX4 Autosoarng Implementation — Defence Technical Report

Fixed-Wing Autosoarng Project

April 2026

## **Contents**

# 1 Introduction

This report describes the inter-process communication (IPC) infrastructure used to command the auto-soaring system from a ROS 2 companion computer to the PX4 flight management unit (FMU). It covers:

- the uORB publish-subscribe middleware;
- the XRCE-DDS bridge that connects ROS 2 to uORB;
- the original messaging approach (boolean flags);
- the redesigned `AutosoaringControl` message using a single enumerated mode integer.

## 2 Original System: uORB and the Absence of Soaring IPC

### 2.1 uORB Publish-Subscribe Middleware

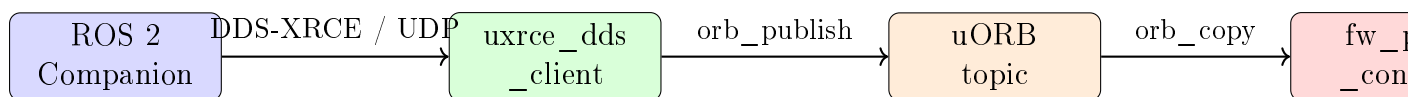
PX4 uses *uORB* (micro Object Request Broker) as its internal IPC bus. It is a strongly-typed, zero-copy, publish-subscribe system running entirely inside the NuttX RTOS process space. Each *topic* is defined by a `.msg` file and compiled into a C struct. Publishers call `orb_publish()`, subscribers call `orb_subscribe()` and `orb_copy()`.

Key properties:

- **Single writer, multiple readers:** uORB supports one active publisher per topic instance. Multiple modules may subscribe.
- **Non-blocking:** `orb_check()` tests for new data without blocking. Blocking waits use `px4_poll()`.
- **Fixed-size structs:** every message has a 64-bit `timestamp` field plus typed payload fields.

### 2.2 XRCE-DDS Bridge

The `uxrce_dds_client` module implements the *DDS-XRCE* (eXtremely Resource Constrained Environments) protocol, bridging uORB topics to a ROS 2 DDS domain:



The bridge is configured in `dds_topics.yaml`. Topics listed under `subscriptions` flow from ROS 2 *into* the FMU (`/fmu/in/`); topics under `publications` flow out.

### 2.3 Original dds\_topics.yaml (no soaring)

In the original *Music* codebase, `dds_topics.yaml` contained no entry for autosoaring. The companion computer had no standardised channel to command soaring behaviour; workarounds relied on MAVLink parameter setting or ad-hoc MAVLink commands.

## 3 Modified System: AutosoaringControl Message Design

### 3.1 Design Rationale: From Two Booleans to One Integer

The first prototype used two boolean fields: `glide_mode_enabled` and `thermal_mode_enabled`. This creates  $2^2 = 4$  possible combinations, two of which are ambiguous:

| glide | thermal | Interpretation                   |
|-------|---------|----------------------------------|
| false | false   | powered flight                   |
| true  | false   | glide mode                       |
| false | true    | thermal mode                     |
| true  | true    | <b>ambiguous — both asserted</b> |

Furthermore, a separate PX4 parameter `FW_GLIDE_MODE` (integer 0 or 1) controlled whether the glide airspeed used a fixed value or the polar-optimal speed, creating a *hidden third dimension* invisible in the uORB message.

The redesign encodes all five states in a single `uint8 soaring_mode` field:

| Value | Constant                            | TECS behaviour     | Navigator state                             |
|-------|-------------------------------------|--------------------|---------------------------------------------|
| 0     | <code>SOARING_OFF</code>            | powered            | unchanged                                   |
| 1     | <code>SOARING_GLIDE_FIXED</code>    | gliding, fixed EAS | <code>AUTO_MISSION</code>                   |
| 2     | <code>SOARING_GLIDE_POLAR</code>    | gliding, polar EAS | <code>AUTO_MISSION</code>                   |
| 3     | <code>SOARING_THERMAL_LOITER</code> | gliding            | <code>AUTO_LOITER</code> (navigator-guided) |
| 4     | <code>SOARING_THERMAL_BANK</code>   | gliding            | <code>AUTO_LOITER</code> (roll override)    |

### 3.2 Complete AutosoaringControl Message

Listing 1: AutosoaringControl.msg (new)

```
uint64  timestamp                # microseconds since boot

uint8    soaring_mode            # see constants below
uint8    SOARING_OFF             = 0
uint8    SOARING_GLIDE_FIXED     = 1
uint8    SOARING_GLIDE_POLAR     = 2
uint8    SOARING_THERMAL_LOITER  = 3
uint8    SOARING_THERMAL_BANK    = 4

# Loiter geometry (mode 3 only)
float32  loiter_radius_m         # [m]; NaN => bank-derived minimum
```

```

float32 loiter_lat          # [deg]; NaN/0 => current position
float32 loiter_lon          # [deg]; NaN/0 => current position

# Optimisation commands (NaN => PX4 parameter default)
float32 airspeed_cmd        # [m/s EAS] mode 1 only
float32 bank_angle_cmd      # [deg] modes 3 and 4
bool    loiter_clockwise    # true=CW, false=CCW

# Metadata
uint8    soaring_phase
uint8 SOARING_PHASE_CRUISE = 0
uint8 SOARING_PHASE_GLIDING = 1
uint8 SOARING_PHASE_SEARCHING = 2
uint8 SOARING_PHASE_CENTERING = 3
uint8 SOARING_PHASE_THERMAL = 4

```

### 3.3 Field Usage Matrix

| Field            | Mode 0 | Mode 1 | Mode 2 | Mode 3 | Mode 4 |
|------------------|--------|--------|--------|--------|--------|
| airspeed_cmd     | –      | ✓      | –      | –      | –      |
| loiter_radius_m  | –      | –      | –      | ✓      | –      |
| loiter_lat/lon   | –      | –      | –      | ✓      | –      |
| bank_angle_cmd   | –      | –      | –      | ✓      | ✓      |
| loiter_clockwise | –      | –      | –      | ✓      | ✓      |
| soaring_phase    | ✓      | ✓      | ✓      | ✓      | ✓      |

### 3.4 Registration in dds\_topics.yaml

Listing 2: Addition to dds\_topics.yaml

```

subscriptions:
  ...
  - topic: /fmu/in/autosoaring_control
    type: px4_msgs::msg::AutosoaringControl

```

This single line enables the bridge to forward ROS 2 messages published on `/fmu/in/autosoaring_control` into the FMU uORB bus.

### 3.5 Staleness Watchdog

A companion-silence detector is implemented in `FixedwingPositionControl::Run()`:

$$\Delta t_{\text{silence}} = t_{\text{now}} - t_{\text{last\_recv}} \quad (1)$$

If  $\Delta t_{\text{silence}} > 2\text{ s}$  and `soaring_mode`  $\neq 0$ , the FMU autonomously sets `soaring_mode`  $\leftarrow 0$  (off) and disables gliding in TECS. This prevents the aircraft from remaining in engine-off glide indefinitely if the companion crashes or the DDS link is lost.

### 3.6 CLI Override Path

For ground testing without a companion, the Commander module accepts CLI commands:

```
commander soar:glide      # mode 1
commander soar:polar      # mode 2
commander soar:thermal    # mode 3
commander soar:bank       # mode 4
commander soar:off        # mode 0
```

Each command sends `VEHICLE_CMD_CUSTOM_0` with `param1` equal to the soaring mode integer. The CLI path sets `_soaring_local_override = true`, which bypasses:

- altitude floor check (`FW_ALT_MIN`);
- staleness watchdog (no companion expected);
- DDS inhibit cooldown.

## 4 Summary of Changes

| Aspect           | Original                                                                       | Modified                                            |
|------------------|--------------------------------------------------------------------------------|-----------------------------------------------------|
| Mode encoding    | Two separate booleans + hidden parameter                                       | Single <code>uint8</code> <code>soaring_mode</code> |
| Polar selection  | PX4 <code>FW_GLIDE_MODE</code> parameter                                       | Encoded in <code>soaring_mode = 2</code>            |
| DDS registration | Not present                                                                    | Added to <code>dds_topics.yaml</code>               |
| Turn direction   | Not supported                                                                  | <code>loiter_clockwise</code> bool                  |
| Staleness guard  | Not present                                                                    | 2-second watchdog in <code>Run()</code>             |
| CLI commands     | <code>soar:glide</code> ,<br><code>soar:thermal</code> , <code>soar:off</code> | + <code>soar:polar</code> , <code>soar:bank</code>  |